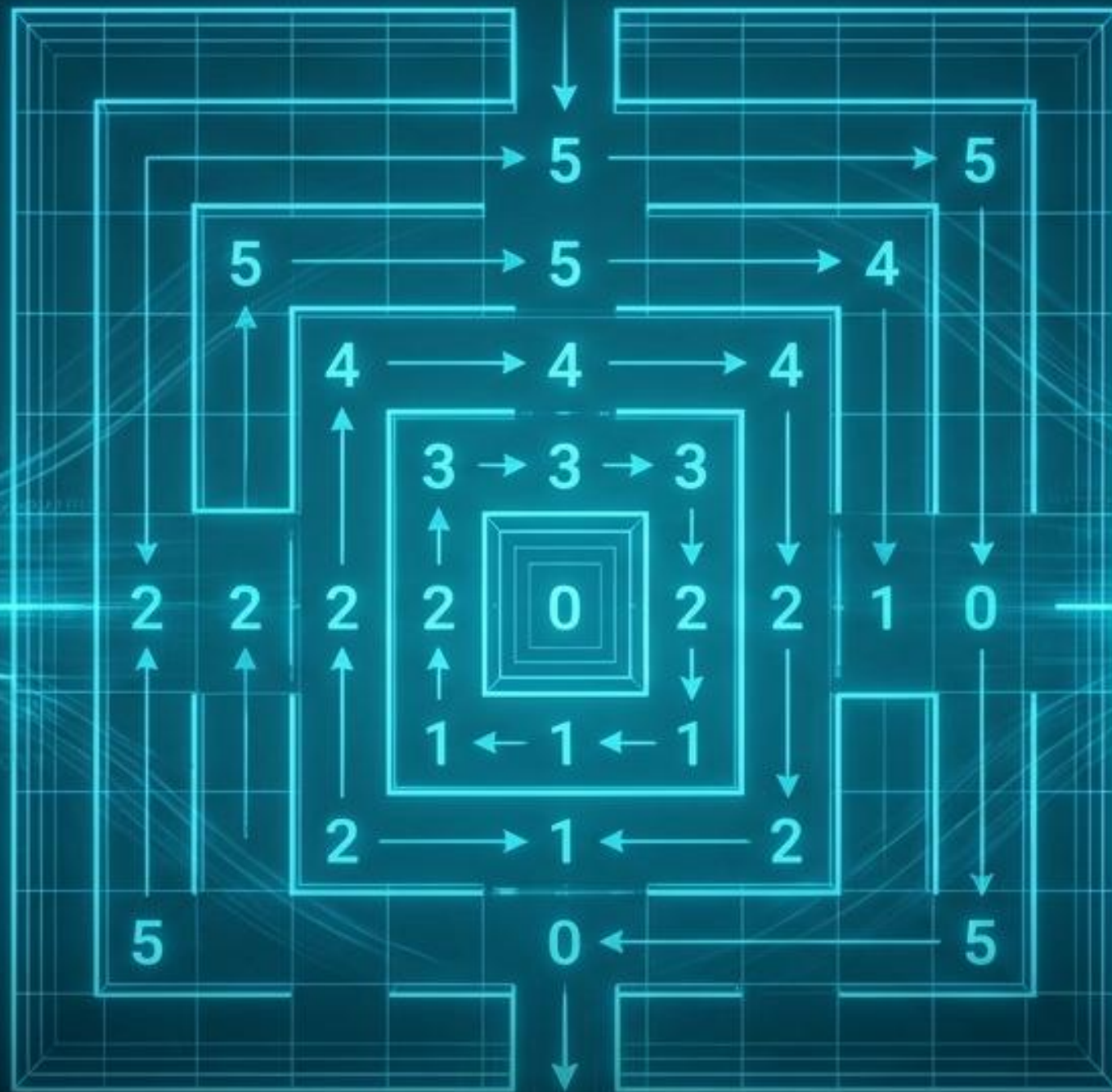


The Blueprint



The Machine



Từ Lý Thuyết Đến Thực Tại: Deep Dive Thuật Toán Floodfill
Kiến trúc Tìm đường, Tối ưu và Quản lý Mê cung trên Micromouse ESP32-S3

Giải Quyết Mê Cung: Toán Học vs. Vật Lý

Tại sao một thuật toán hoàn hảo trên trình giả lập lại thất bại trên robot thực tế?



1. Toán Học Cốt Lõi: Ma Trận Khoảng Cách (Distance Field)

Đổ nước vào mê cung để tạo ra một độ dốc hướng thẳng về đích.

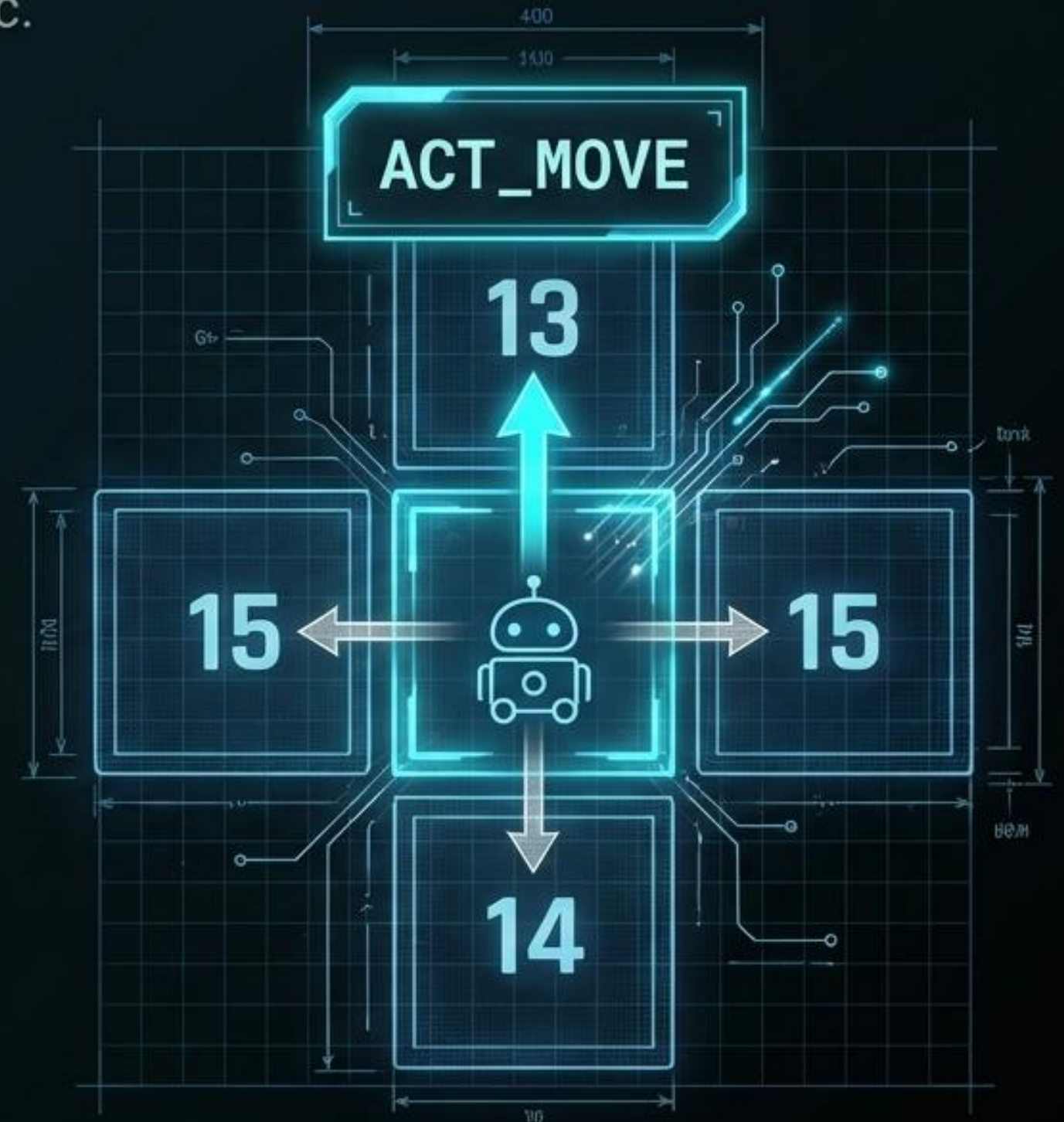
- **Nguyên lý cốt lõi:** Đích đến luôn bị ép giá trị 0. Từ đích, thuật toán loang ra các ô lân cận (không bị cản bởi tường), tăng dần giá trị khoảng cách (+1).
- **Bản đồ Trọng số (Weight Map):** Thao tác này tạo ra một độ dốc toán học. Bất kể vị trí hiện tại của robot ở đâu, nó không cần biết đường đi tổng thể, nó chỉ cần biết làm sao để bước xuống dốc.



1. Ra Quyết Định: Trôi Theo Dòng Nước

Chu trình 4 bước để chọn lối đi hoàn hảo tại mỗi ô rời rạc.

1. **Cảm nhận (Observe):** Đọc cảm biến TOF để cập nhật các bức tường vừa nhìn thấy.
2. **Tính toán lại (Recalculate):** Chạy lại toàn bộ ma trận Floodfill. Một bức tường mới có thể chặn đường đi cũ, làm thay đổi toàn bộ độ dốc của mê cung.
3. **Quét lân cận (Scan):** Đánh giá giá trị của các ô mở xung quanh (Trái, Trước, Phải, hoặc Sau nếu là ngõ cụt).
4. **Hành động (Execute):** Xuất ra lệnh điều hướng hướng tới ô có giá trị nhỏ nhất.
 - Bình thường: Lệnh `ACT_MOVE` đi thẳng, rẽ trái, rẽ phải.
 - Ngõ cụt: Lệnh `ACT_TURN_180` để xoay ngược lại và thoát ra.



2. Quản Lý Bản Đồ: Trái Tim FloodFillExplorer

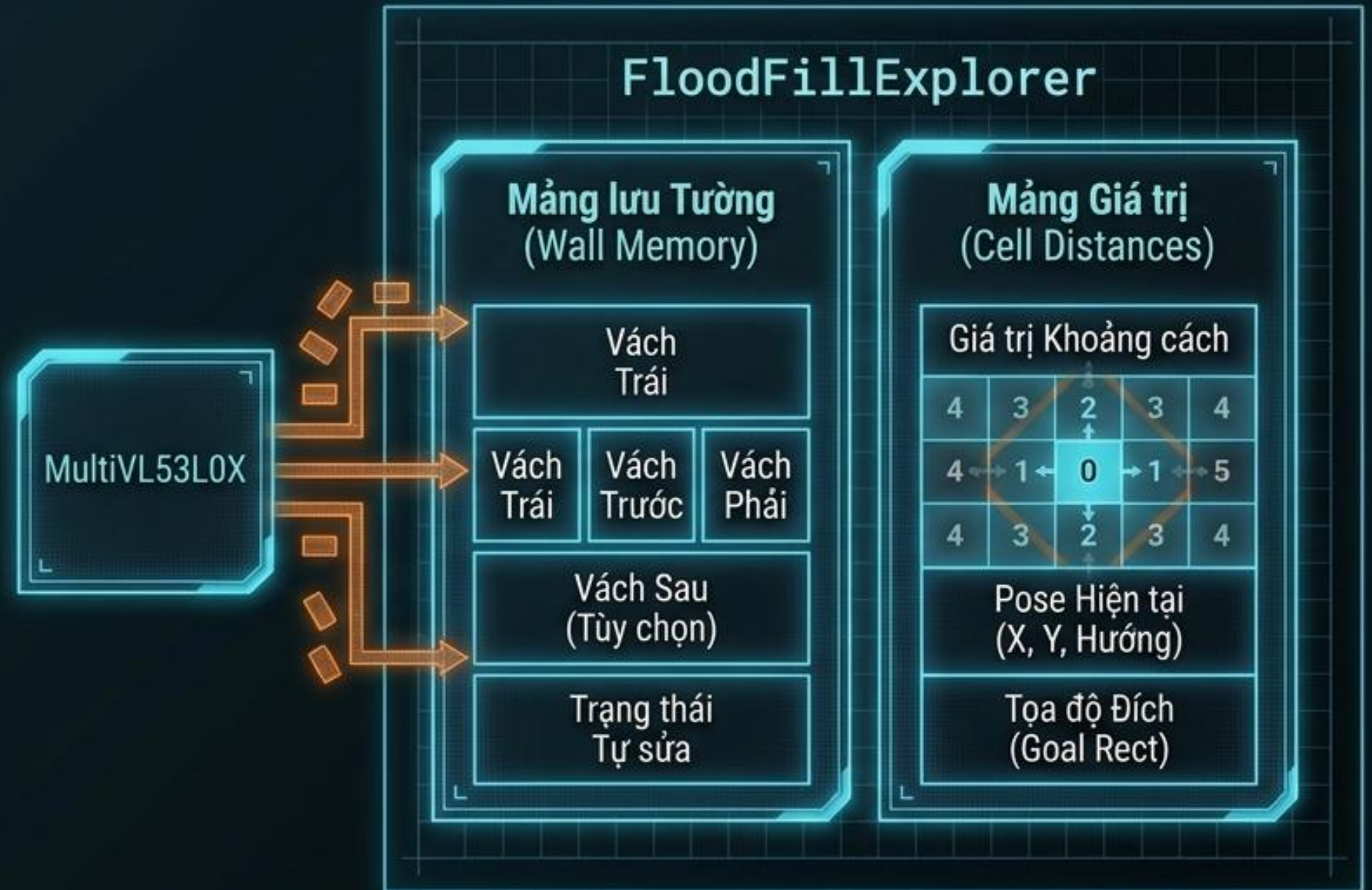
Cách hệ thống ghi nhớ và tự sửa sai thế giới xung quanh nó.

Không gian Rời rạc (Discrete Cells): Hệ thống không dùng tọa độ mm tự do. Môi trường được định nghĩa bằng các ô logic (cell) và tọa độ nguyên.

Cấu trúc Lưu trữ:

- Ghi nhớ các vách ngăn (Trái, Phải, Trước) cho từng tọa độ.
- Quản lý tọa độ hiện tại (Pose) và Tọa độ đích (Goal Rect).

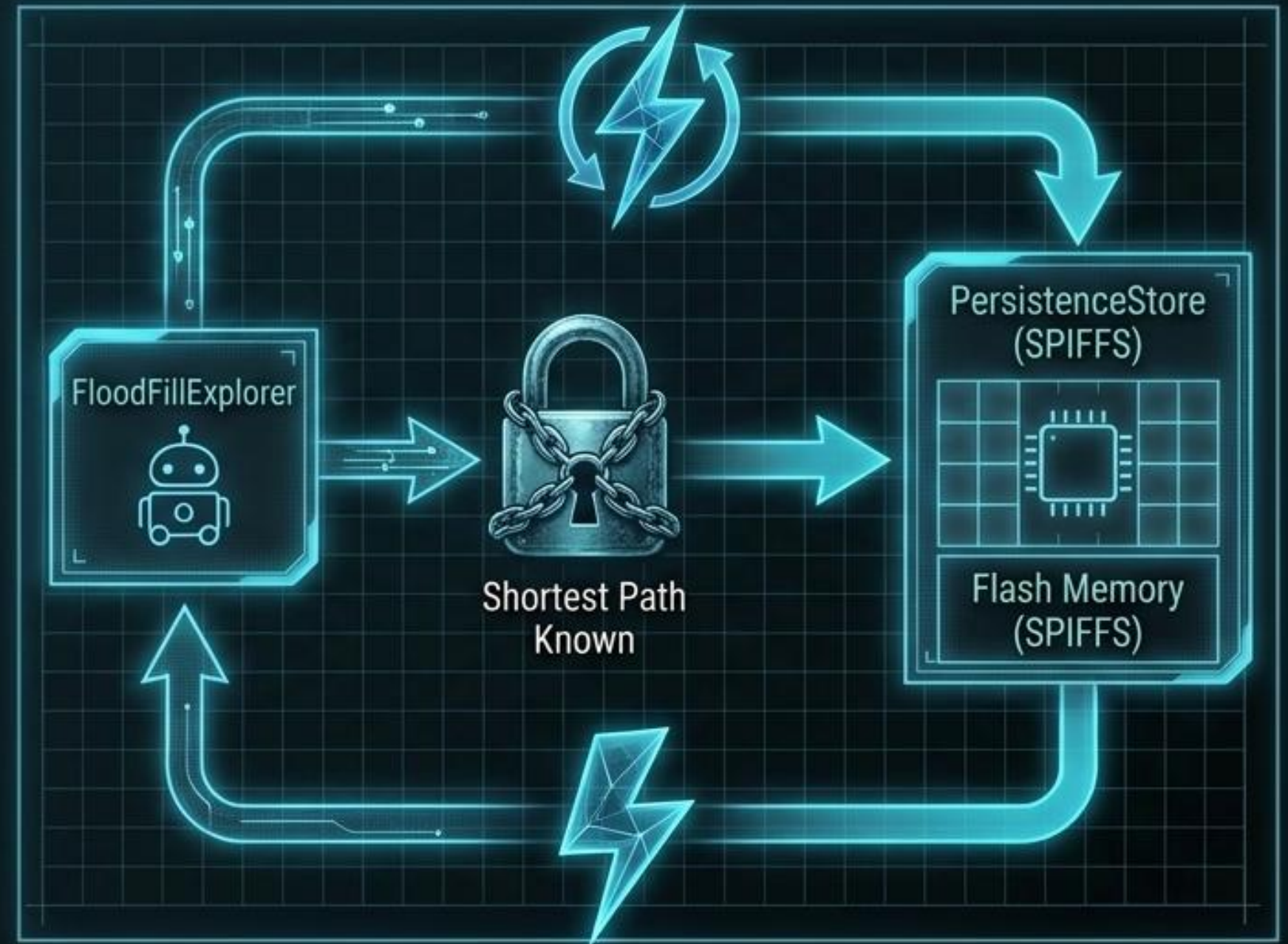
Cơ chế Tự sửa sai (Self-Correction): Chỉ tin vào sự thật vật lý. Các tín hiệu rỗng/xa (far/open) từ cảm biến TOF có thể ghi đè và xóa các bức tường ảo do nhiễu sinh ra trước đó, giúp robot khôi phục đường đi khi đi lại đường cũ.



2. Ký Ước Bền Vững: Lưu Trữ Vào Flash (SPIFFS)

Đảm bảo robot không bao giờ quên đường đi tối ưu ngay cả khi mất điện.

- **Vấn đề:** Trạng thái Floodfill lưu trong RAM sẽ mất sạch nếu reset ESP32-S3.
- **Giải pháp với SPIFFS:** Khi phát hiện đường đi tối ưu, PersistenceStore.cpp ghi trực tiếp bản đồ tường (wall map) vào phân vùng Flash.
- **Khôi phục Tự động:** Lúc boot, hệ thống tự động nạp lại bản đồ từ SPIFFS. Đèn LED chuyển Trắng (Idle Ready), sẵn sàng chạy Speedrun ngay lập tức.
- **Thiết kế An toàn:** Hệ thống chỉ lưu cấu trúc tường. Tọa độ xuất phát (Pose) và Đích (Goal) tuyệt đối không lưu vào SPIFFS để tránh nhầm lẫn nếu đặt robot ở vị trí khác.



3. Khoảng Trống Thực Tế: Ảo Tưởng Của Giả Lập

Quy tắc vàng: Đừng tin vào suy nghĩ của bạn cho đến khi cơ thể bạn làm được.

Lỗi Thường Gặp

Trên trình giả lập (simulator), dịch chuyển xảy ra tức thì. Trên ESP32-S3, một ô dịch chuyển mất hàng trăm mili-giây và có nguy cơ kẹt bánh, trượt gia tốc.



Kiến trúc Chuẩn

Giải pháp kiến trúc: Tách biệt hoàn toàn phần mềm lập kế hoạch (Planner) và cơ bắp thực thi (Executor).



3. Hệ Điều Hành Robot: Phân Chia Quyền Lực

Hợp đồng hệ thống (System Contract) chuẩn giáo dục của mã nguồn.

Sự phân lập này ngăn ngừa các lỗi trạng thái (state bugs) khó gỡ lỗi nhất trong Robotics, đảm bảo hệ thống luôn có một nguồn chân lý duy nhất (Single Source of Truth).

	Planner (FloodFillExplorer)	Executor (MotionController)
Vai trò	Đầu não chiến lược (Bộ não)	Xử lý chuyển động vật lý (Cơ bắp)
Input	Bản đồ tường, Ma trận Floodfill	Lệnh nguyên thủy, Xung Encoder, PID
Output	Hành động tiếp theo (ACT_MOVE, ACT_TURN_90)	Kiểm soát động cơ thực tế, Phanh (Brake)
Quản lý Tọa độ	Bị cấm tự cập nhật tọa độ	Không quan tâm tọa độ logic
Tính chân thực	Chỉ đưa ra Giả định đường đi	Quyết định thành công/thất bại vật lý

3. Cơ Chế Bắt Tay (The ACK Protocol)

Quy trình vòng kín an toàn tuyệt đối để cập nhật trạng thái mê cung.

Robot chỉ cập nhật Pose (Vị trí/Hướng) và Tường sau khi quá trình di chuyển vật lý hoàn tất hoàn toàn.

Bức tường phía sau cũng được tính toán lại sau khi kết thúc lệnh rẽ thông qua hàm snapCenter().

6. **Xác nhận (ACK):** AppRuntime gửi ACK cho Planner để chính thức cập nhật Tọa độ (Pose).

5. **Cảm nhận sự thật:** Cảm biến TOF quét và cập nhật Tường thực tế tại vị trí vật lý mới.



1. **Kế hoạch:** Planner quyết định và gửi lệnh ACT_MOVE.

2. **Chuyển giao:** AppRuntime điều phối lệnh cho MotionController.

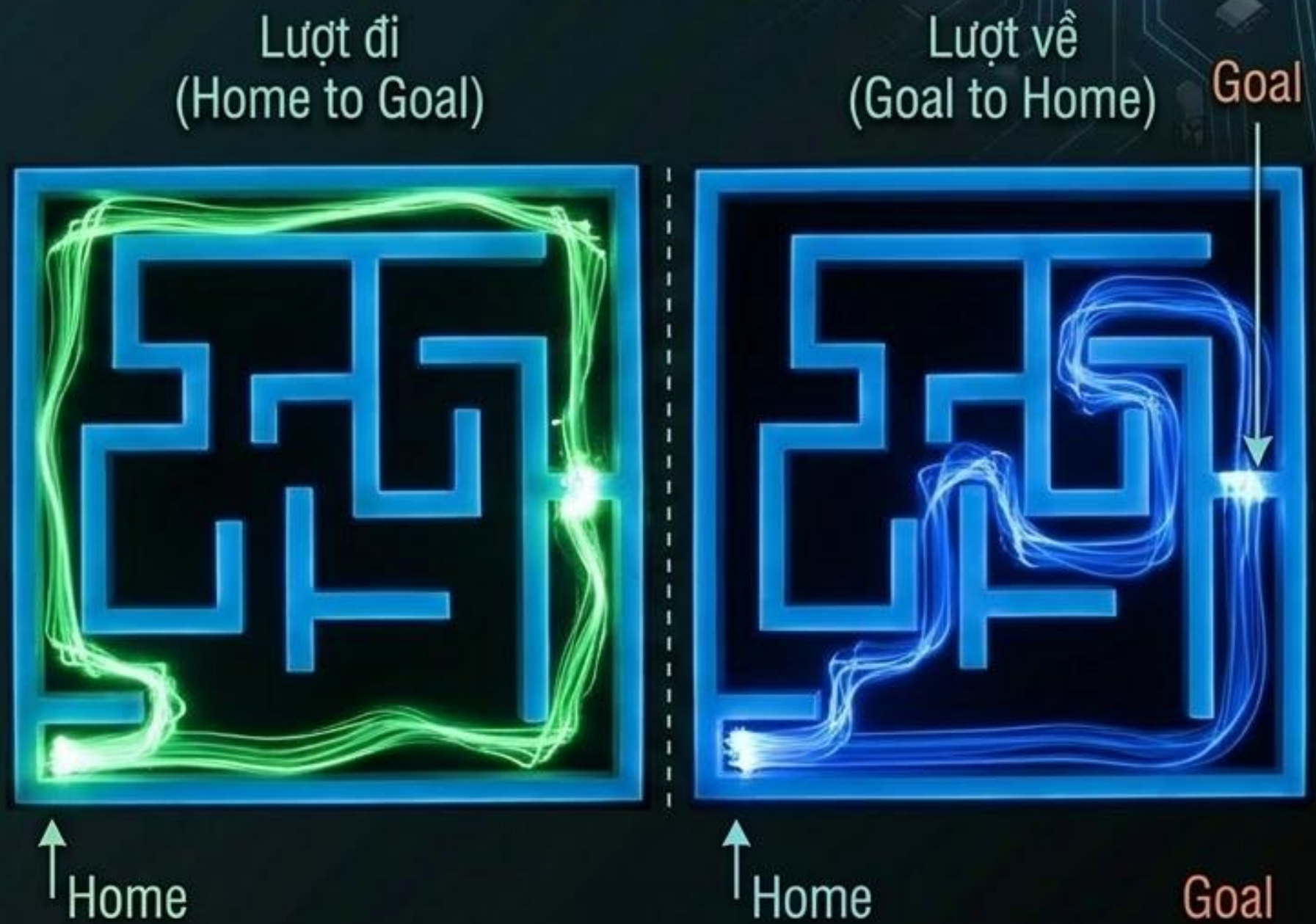
3. **Thực thi:** Robot di chuyển vật lý (Bánh xe quay).

4. **Dừng cứng (Hard-stop):** Di chuyển xong, MotionController áp dụng phanh và gửi tín hiệu Hoàn thành.

4. Khám Phá Liên Tục: Chế độ Đảo Đích

Cơ chế CONTINUE_AFTER_GOAL (Goal-to-Home Swapping).

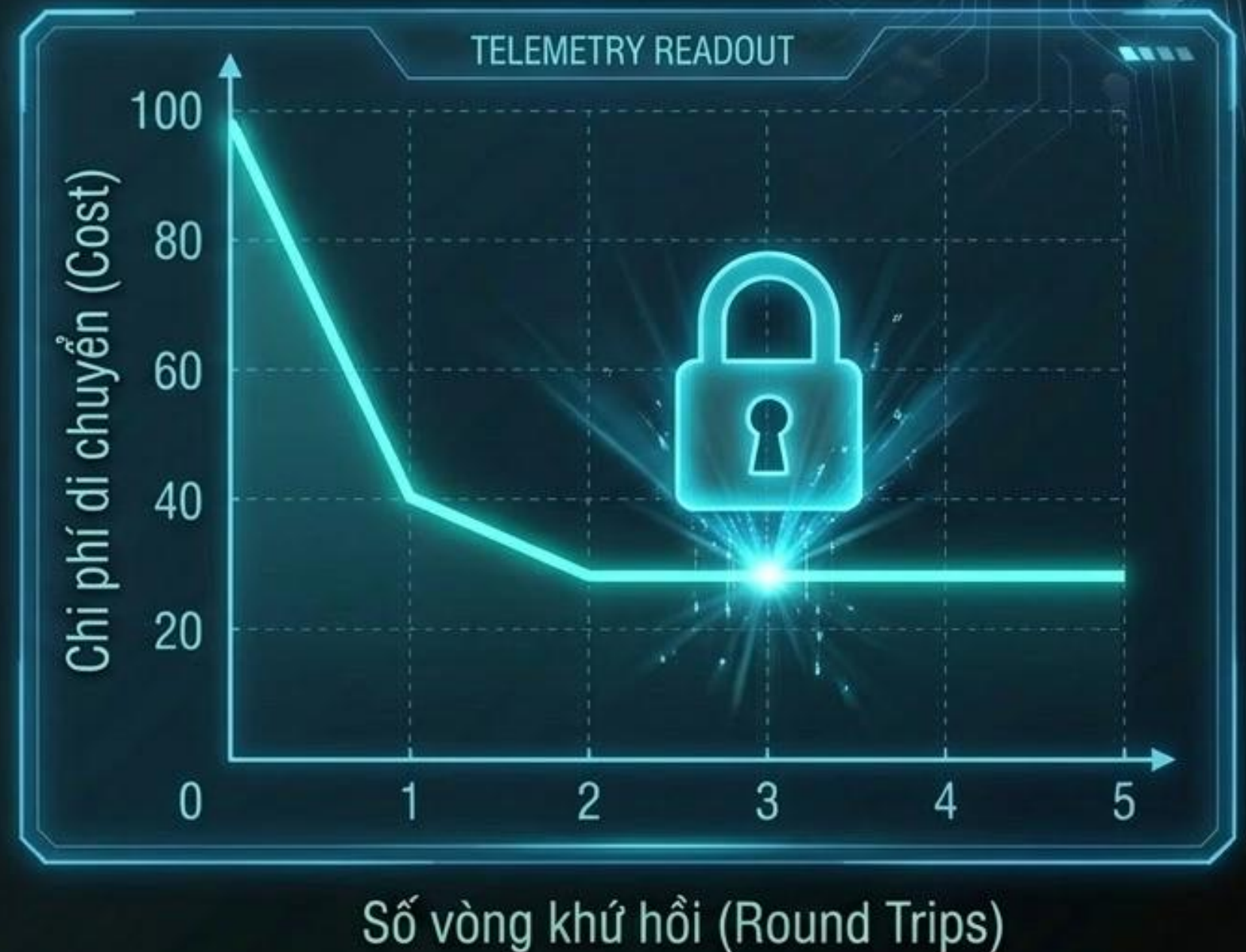
- Vấn đề: Đường đi đầu tiên chạm đích thường không phải là đường ngắn nhất (do robot có xu hướng đi men tường).
- Giải pháp Cấu hình: Bật cờ CONTINUE_AFTER_GOAL trong Config.h.
- Cơ chế Swapping: Khi chạm Đích, FloodFillExplorer không dừng lại. Nó lật ngược mục tiêu: Đích biến thành Điểm xuất phát (Home), và Home biến thành Đích.
- Kết quả: Robot tự động chạy khứ hồi, khám phá thêm các nhánh rẽ và tự động tối ưu hóa lại ma trận khoảng cách mà không cần can thiệp thủ công.



4. Khi Nào Mê Cung Đã Được Giải?

Chốt khóa đường đi bằng hằng số SHORTEST_PATH_STABLE_ROUND_TRIPS.

- **Best-known Cost:** Runtime liên tục theo dõi chi phí di chuyển tối ưu nhất từ Home đến Goal sau mỗi vòng.
- **Bộ đếm Ổn định:** Sau một vòng khứ hồi (Round Trip) thành công, nếu Cost không giảm thêm, bộ đếm ổn định sẽ tăng lên 1.
- **Khóa Mục tiêu:** Khi bộ đếm đạt tới giới hạn cấu hình SHORTEST_PATH_STABLE_ROUND_TRIPS (thường = 1).
- **Hành động cuối:** Ngừng khám phá -> In thông báo Shortest path known -> Ghi vĩnh viễn vào SPIFFS -> Chuyển sang nhấp nháy đèn Trắng.



5. Tầm Nhìn Xuyên Thấu: Công Cụ Web UI

Trạm điều khiển từ xa qua Wi-Fi (Cổng 81).

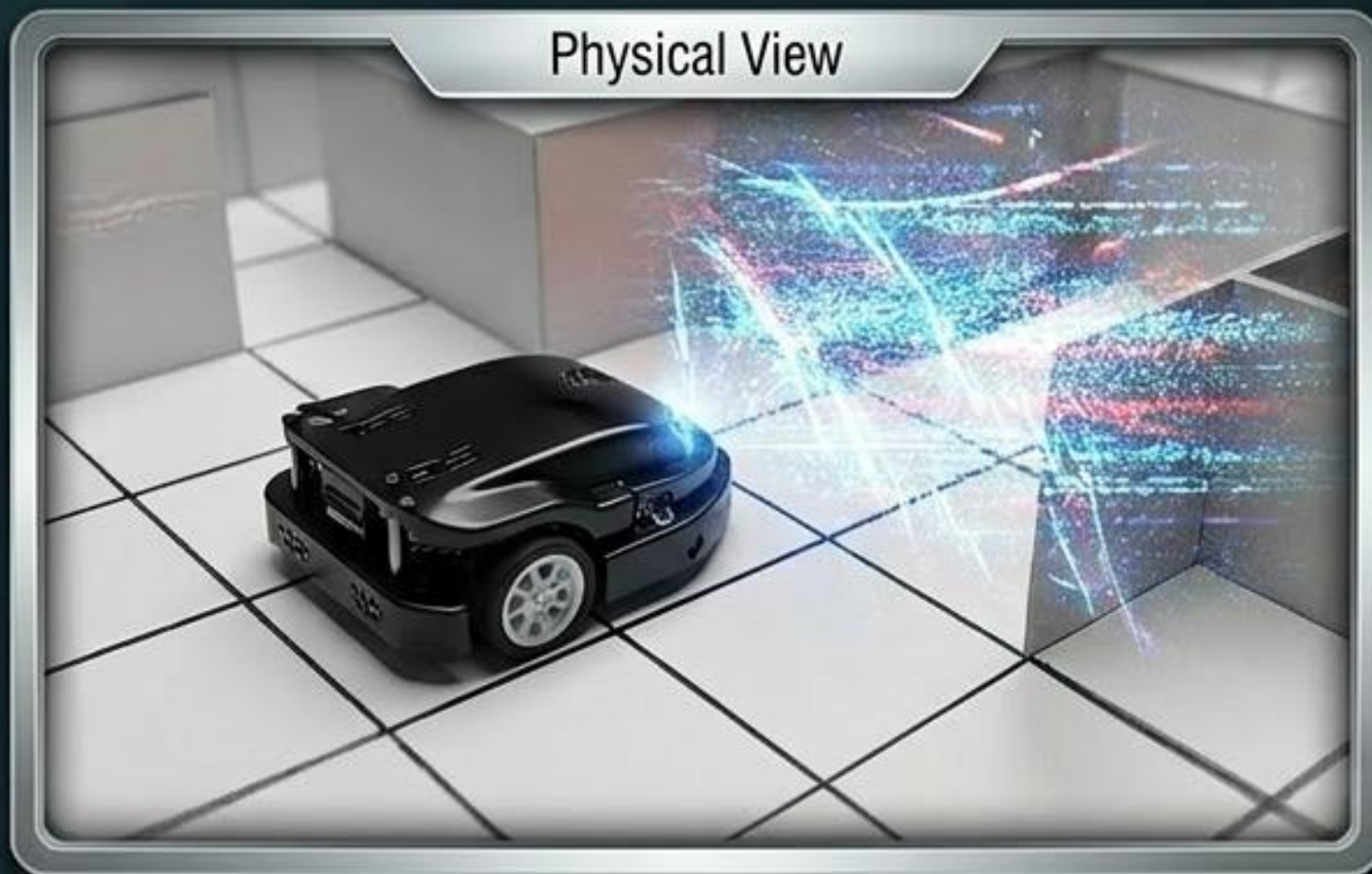
- **Bản chất:** Web Explorer không phải nguồn chân lý vật lý, mà là một công cụ gỡ lỗi (Debug Instrument) để nhìn thấu bộ não lập kế hoạch.
- **Công nghệ WebSocket:** Giao tiếp thời gian thực hai chiều. Chấm dứt hiện tượng giật lag do trình duyệt polling HTTP liên tục.
- **Hiển thị Live:** Quan sát ma trận khoảng cách Floodfill liên tục thay đổi hình dạng mỗi khi robot phát hiện thêm một bức tường mới.
- **Kiểm soát:** Tích hợp nút Step/Run để kiểm tra luồng runtime chờ ACK (AutoACK off) hoặc mô phỏng logic thuần túy (AutoACK on).



5. Tư Duy vs. Thực Tế: Sức Mạnh Debugging

Sử dụng Cổng 81 để bắt bệnh cảm biến và encoder.

Sự khác biệt giữa những gì bạn thấy và những gì robot thấy, là cội nguồn của mọi lỗi. Dùng Web UI để đối chiếu:



- Tường giả (Phantom walls): Nhiều cảm biến tạo ra tường ảo -> Floodfill chọn đường sai vòng vào -> Xử lý bằng cách tinh chỉnh ngưỡng (thresholds) trong Config.h.
- Mất Pose (Lạc tọa độ): Robot bị kẹt bánh, quay 90 độ vật lý nhưng encoder đếm thiếu -> Web UI hiển thị hướng sai lệch so với thực tế -> Dấu hiệu cần cân chỉnh lại turnTicks90 thay vì sửa thuật toán.

Tổng Hợp: Chu Trình Khám Phá Hoàn Chỉnh

Từ Cảm biến đến Thuật toán, tới Bộ nhớ vĩnh cửu.

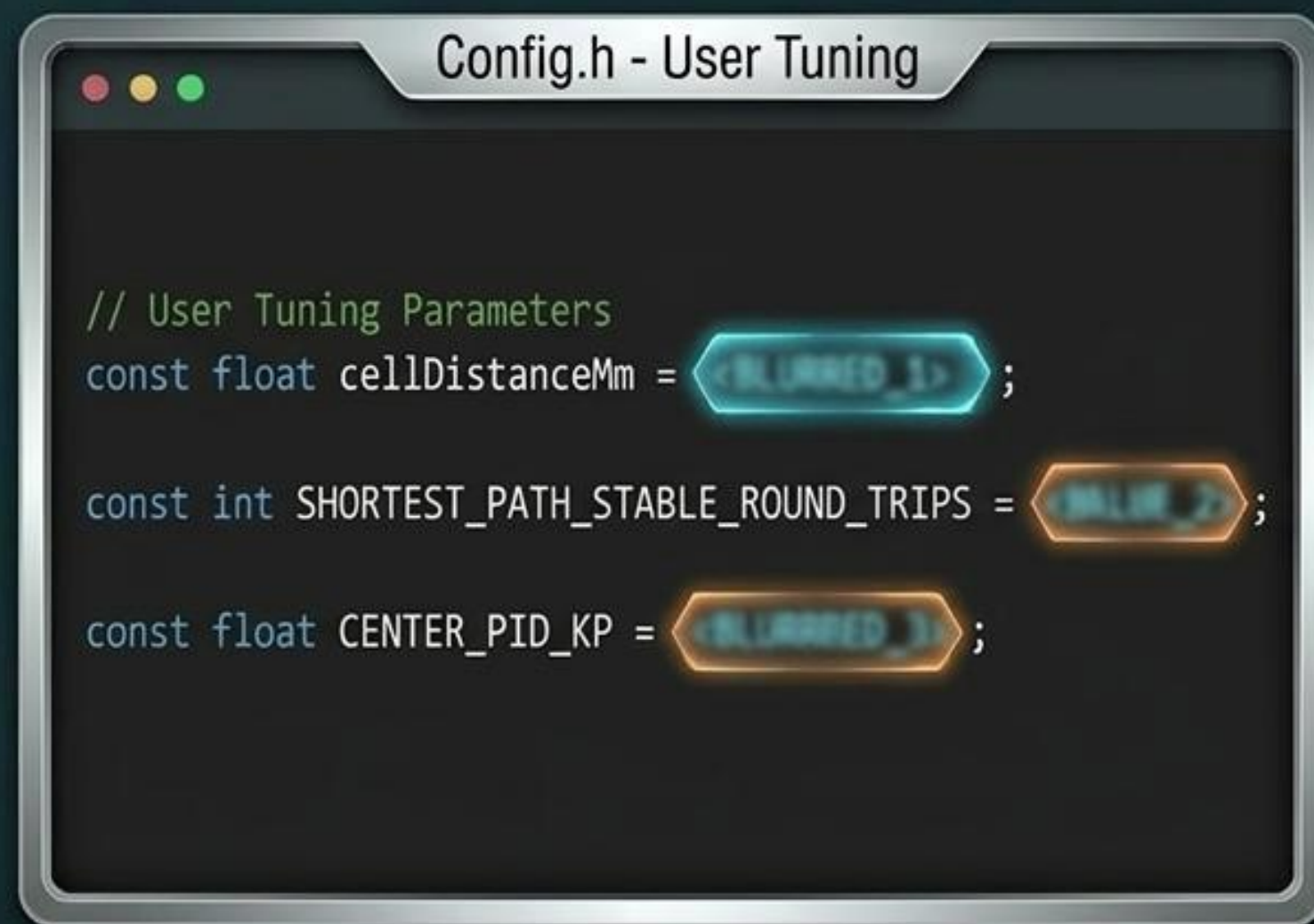
Floodfill trên ESP32-S3 không chỉ là một thuật toán trên giấy. Nó là một thực thể sống, được điều phối bởi kiến trúc phần mềm nghiêm ngặt. Sự tách biệt nhiệm vụ là chìa khóa để thuật toán hoạt động đáng tin cậy.



Kết Luận & Bước Tiếp Theo

Làm chủ mê cung nằm trong tay bạn. Biến lý thuyết thành thực tại.

Một thuật toán hoàn hảo vẫn cần một nền tảng phần cứng đáng tin cậy.



Khuyến nghị thực hành (Bring-up Order):

1.  Ưu tiên tuyệt đối độ ổn định (Reliability) trước khi đẩy cao tốc độ (Aggressive Optimization).
2.  Tinh chỉnh di chuyển 1 ô (cellDistanceMm) cực chuẩn xác.
3.  Thiết lập ngưỡng SHORTEST_PATH_STABLE_ROUND_TRIPS phù hợp độ khó mê cung.
4.  Mở Cổng 81, theo dõi dòng nước Floodfill lan tỏa và tinh chỉnh ngưỡng cảm biến Tof.

Hãy mở Config.h và bắt đầu chinh phục mê cung!